
Object Detection Pipeline on Kubernetes

CSC 581 – Cloud Computing | Fall 2026

Logan Yates

West Chester University of Pennsylvania

September 2026

Chapter 1: Vision

1.1 Overview

For every second, billions of cameras around the world capture images that no human will ever review. Security feeds go unwatched. Medical scans wait in queues. Retail shelves go unaudited. The bottleneck is not the hardware; it is the intelligence at scale. Object detection, the ability to pinpoint and name every meaningful element within an image, has become one of the most transformative capabilities in modern software. It powers the defect spotting on factory assembly lines, the shoplifting alerts in retail stores, and the crowd-density monitoring in public spaces.

Deploying object detection in production is not simply a matter of running a model on a laptop. Real-world systems must handle scale under load, version their components independently, and guarantee that no result is ever silently lost. These are infrastructure problems and Kubernetes is the industry's answer to them.

This project delivers a fully containerized, cloud-native system that accepts a raw image from a user, preprocesses it into the precise data block format required by a neural network, runs it through a pre-trained **MobileNet-SSD** model capable of detecting 90 distinct object classes, and persistently stores the original image, the processed data block, and a structured JSON result containing every detected object's label, confidence score, and bounding box. The system is designed as a set of loosely coupled microservices, each with a separate Kubernetes Pod communicating over encrypted internal ClusterIP Services, with exactly one external entry point exposed to the world.

The architectural choices are deliberate. A single NodePort gateway keeps the attack surface minimal. Pinned container image versions make every deployment reproducible. A shared **PersistentVolumeClaim (PVC)** with a structured directory layout ensures that no artifact is lost even if a Pod crashes mid-pipeline. And the inference Pod, the compute heart of the system, is the designated horizontal scale target: add more replicas and throughput scales linearly, without touching any other component.

MobileNet-SSD was selected deliberately. At roughly 20 MB, it is small enough to load in seconds, fast enough to run on CPU without a GPU, and accurate enough to detect real objects in real images.

1.2 Architecture

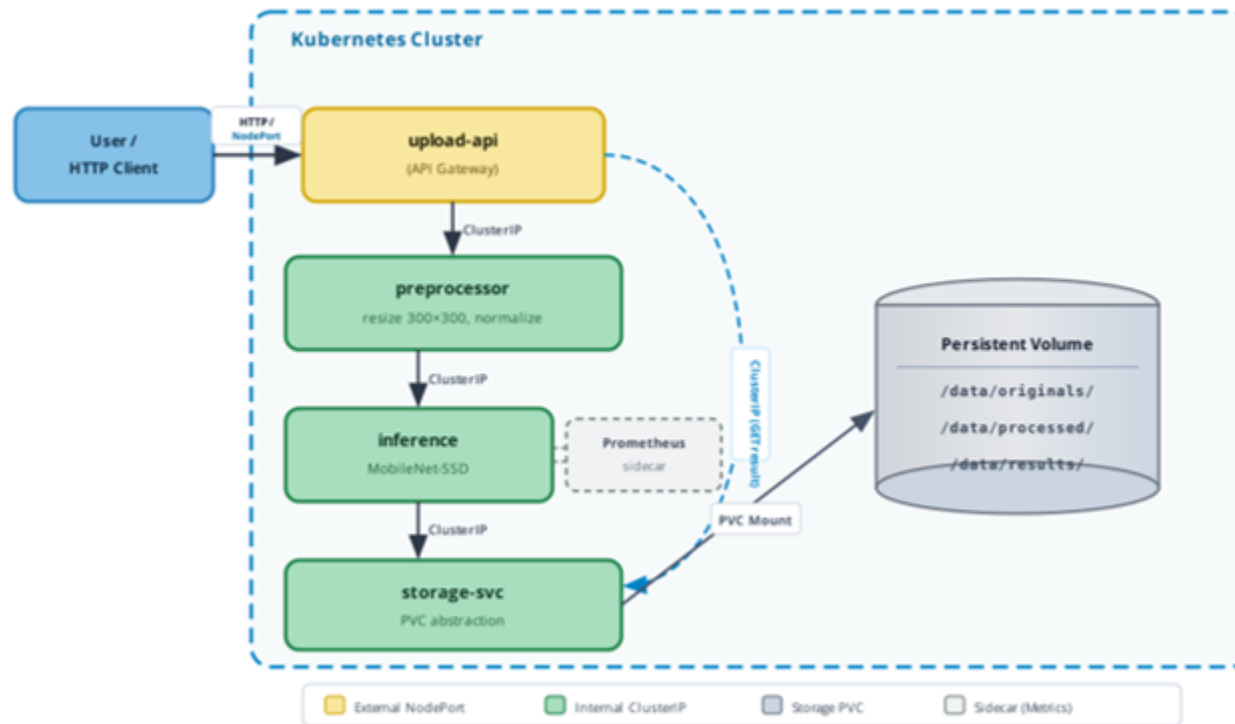


Figure 1 - Architecture: Object Detection Pipeline on Kubernetes

The architecture strictly enforces project constraints and cloud-native design patterns. In accordance with requirements, the system exposes exactly one external entry point: the upload-api Service bound via NodePort, which shields internal pipeline components from unauthorized direct access. All inter-service communications occur entirely over private Kubernetes ClusterIP Services using internal CoreDNS names, making intermediate stages completely invisible from outside the cluster. The inference Pod serves as the designated Horizontal Pod Autoscaler (HPA) scale target because it is compute-intensive and entirely stateless enabling dynamic replica scaling based on CPU utilization without distributed coordination. A co-located Prometheus sidecar container exports latency and throughput metrics on port 9090, demonstrating the multi-container Pod pattern. Durable state is governed by a shared PersistentVolumeClaim organized into /data/originals/, /data/processed/, and /data/results/, where UUID-based naming prevents write collisions under concurrent uploads. Finally, pinned container image tags ensure clean reprovisioning via kubectl delete -f . && kubectl apply -f . with zero configuration drift.

Chapter 2: Implementation Proposal

2.1 Use Cases & User Stories

Story 1 - Upload & Receive ID: As a user, I upload a JPEG image via a REST API and immediately receive a UUID confirmation, so that I can retrieve my object detection result asynchronously without blocking my application.

Story 2 - Retrieve Detection Results: As a user, I query the API with my UUID and receive a structured list of every detected object label, confidence score, and bounding box coordinates so that I can overlay annotations on the original image or feed results into a downstream system.

Story 3 - Operator Scale-Out: As a system operator, I horizontally scale the inference Deployment during peak load periods, so that detection throughput increases proportionally without redeploying or reconfiguring any other service in the pipeline.

2.2 Data-Flow Diagram



Figure 2 - Data-Flow Diagram: *upload* → *preprocessing* → *inference* → *storage* → *retrieval*

The data pipeline operates entirely asynchronously to ensure fast user response times and robust fault tolerance. When a client submits an image at Step 1, upload-api validates payload constraints and returns an immediate 202 Accepted containing a tracking UUID. Steps 2, 3, and 4 proceed sequentially behind the scenes across internal microservices without blocking the client. When inference finishes and results are written to disk, the client retrieves the complete detection payload by polling Step 5 with its assigned UUID.

2.3 Kubernetes Component Design

The pipeline is decomposed into four Deployments plus one sidecar container, each with a single responsibility.

Table 1: Kubernetes Pipeline Workloads

Component	Kind	Replicas	Pinned Image Tag	Role
upload-api	Deployment	1	upload-api:1.0.0	External REST API; UUID assignment; result retrieval
preprocessor	Deployment	1	preprocessor:1.0.0	Resize to 300×300, normalize to [0,1]
inference	Deployment	1–N (HPA)	inference:1.0.0	MobileNet-SSD detection forward pass
storage-svc	Deployment	1	storage-svc:1.0.0	CRUD abstraction layer over shared PVC
prometheus-sidecar	Sidecar (in inference Pod)	N/A	prom/pushgateway:v1.6.2	Exposes /metrics port 9090

upload-api: External FastAPI gateway; validates JPEG/PNG (≤ 2 MB), assigns UUIDv4, streams raw file to storage-svc, dispatches preprocessing, and serves client result polling.

preprocessor: Input transformation service; pulls raw asset via UUID, resizes to 300×300, normalizes pixel values to [0,1], and saves binary NumPy tensor (.npy) to /data/processed/.

inference: Computational core with pinned PyTorch/torchvision; runs MobileNet-SSD forward pass on CPU, applies 0.5 confidence threshold, and emits structured detection JSON to storage-svc.

storage-svc: Internal CRUD REST API over shared PVC; manages file I/O, prevents lock collisions, and isolates storage backend from compute pods.

Multi-container Pod note: The inference Pod is the only multi-container Pod in the pipeline, co-locating MobileNet-SSD with a Prometheus sidecar exporting /metrics on port 9090 demonstrating the sidecar pattern per course requirements.

2.4 Service Choices & Scaling Target

Table 2: Kubernetes Service Configurations

Service	Type	Port	Justification
upload-api-svc	NodePort	30080	Single external endpoint which satisfies the one-entrypoint constraint
preprocessor-svc	ClusterIP	8081	Internal pipeline stage only
inference-svc	ClusterIP	8082	Internal pipeline stage only
storage-svc	ClusterIP	8083	Internal I/O abstraction but not externally reachable

The inference Deployment is designated as the HPA scale target, governed by the following HorizontalPodAutoscaler configuration:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: inference-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: inference
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Scaling Rationale: The inference Pod is completely stateless and compute-heavy, making it the primary throughput bottleneck in the pipeline. All other Deployments remain at 1 replica because their lightweight asynchronous I/O and preprocessing workloads do not bottleneck under ≤ 2 MB inputs. Dynamic HPA scaling ensures that CPU resources scale out during traffic bursts and release cleanly when idle.

2.5 Persistence Plan

One PVC: pipeline-pvc accessModes: ReadWriteMany, storage: 5Gi, hostPath for local development. Directory layout:

```
/data/  
originals/{uuid}.jpg  <- raw user upload (JPEG/PNG <= 2 MB)  
processed/{uuid}.npy  <- normalized 300x300 float32 tensor  
results/{uuid}.json   <- { detections: [{label, confidence, bbox}] }
```

Table 3: Pipeline Artifact Persistence Strategy

Artifact	Format	Written By	Read By	Directory
Original image	.jpg / .png	upload-api	preprocessor	/data/originals/
Normalized tensor	.npy	preprocessor	inference	/data/processed/
Detection result	.json	inference	upload-api (retrieval)	/data/results/

UUID filenames prevent write collisions during concurrent uploads. storage-svc is the sole component with PVC access to all others call it via ClusterIP to ensure unified file management.

2.6 API Contracts

POST /images: Upload an image

```
Content-Type: multipart/form-data  
Body field:  file = <image bytes> (JPEG or PNG, max 2 MB)  
  
202 Accepted:  
{ "id": "550e8400-e29b-41d4-a716-446655440000", "status": "processing" }  
  
413 Payload Too Large: image exceeds 2 MB limit  
415 Unsupported Media Type: file is not JPEG or PNG
```

GET /images/{id}/result: Retrieve detection result

200 OK:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-09-25T14:32:00Z",
  "detections": [
    { "label": "person", "confidence": 0.94, "bbox": [120, 45, 80, 210] },
    { "label": "bicycle", "confidence": 0.76, "bbox": [300, 90, 150, 180] }
  ]
}
```

202 Accepted: { "status": "processing" } <- inference not yet complete

404 Not Found: { "error": "not found" }

2.7 Risk & Test Plan

Top Risks

Table 4: Pipeline Risks and Mitigations

1	Risk	Likelihood	Mitigation
1	Model checkpoint (~20 MB) slows Pod startup	Medium	initContainer pre-pulls weights; readinessProbe delays traffic until ready
2	PVC write contention under concurrent uploads	Medium	UUID filenames eliminate collisions; storage-svc serializes writes if needed
3	Large images exhaust memory or exceed timeout	High	2 MB limit enforced at API layer; resources.limits set on inference Pod
4	Stale PVC data after kubectl delete && apply	Low	Namespace-scoped PVC; documented teardown script included in repo
5	PyTorch / torchvision version drift	Low	All image tags and requirements.txt pinned to exact versions
6	Low-confidence detections cluttering results	Medium	Configurable confidence threshold (default 0.5) applied at inference time

Test Matrix

Table 5: System Verification Test Matrix

Test Case	Input	Expected Output	Method
Happy path	Valid JPEG, 100 KB, visible objects	HTTP 202 → poll → 200 with detections array	curl + kubectl port-forward
Empty scene	Plain white JPEG	HTTP 200, “detections”: []	curl
Large image	PNG, 5 MB	HTTP 413	curl
Invalid format	PDF file	HTTP 415	curl
10 concurrent uploads	10 valid JPEGs	10 unique IDs, all eventually 200	Parallel curl loop
Clean reprovision	kubectl delete -f . && kubectl apply -f .	All Pods reach Running; no residual errors	kubectl get pods
HPA scale-out	Sustained CPU load via concurrent requests	New inference replica observed	kubectl get hpa

2.8 Team Charter

Table 6: Team Roles & Responsibilities

Role	Owner
Architecture & Design	Logan Yates
Kubernetes Configuration	Logan Yates
Service Implementation	Logan Yates
Testing & Verification	Logan Yates
Documentation	Logan Yates

Meeting Cadence: Weekly self-review against milestone checklist

Definition of Done: A milestone is complete when all the following are true:

- All Pods reach Running state after a clean `kubectl delete -f .` && `kubectl apply -f .`
- The happy-path test passes end-to-end via `curl` + `kubectl port-forward`
- Edge case tests (large image, invalid format) return correct HTTP status codes
- All deliverable files are committed to the public GitHub repository

Logan Yates

Email: loganllyates@yahoo.com

LinkedIn: <https://www.linkedin.com/in/logan-yates-50771b256/> | GitHub: <https://github.com/LegionYata>

EDUCATION

West Chester University

Bachelor's Degree in Computer Science (Cloud Engineering) & Minor in Music Production

GPA: 3.5

West Chester, PA

August 2022 - May 2026

Honors:

Cum Laude

Dean's Lists (Fall 2022, Spring 2023, Spring 2026) Honor Roll

Scholarships:

The Woodman McIntosh STEM Opportunity Scholarship Fund

The College of the Sciences and Mathematics (CSM) Scholarship

Relevant Coursework:

Computer Science III, Data Structures and Algorithms, Modern Web Applications, Operating Systems, Modern Web Applications, Database Management Systems, Data Communications & Networking, Data Analytics, Software Engineering, Intro. to Cloud Computing.

CAMPUS INVOLVEMENT

Competitive Programming Club

Member

September 2025 – May 2026

- Participated in monthly programming competitions with club members to solve difficult coding problems.
- Discussed strategies to carry through coding problems much more efficiently.
- Conducted retrospective analysis of archived problems to refine Dynamic Programming and optimize Graph Algorithms.

Computer Science Club

Member

August 2022 – May 2026

- Engaged in monthly Lunch & Learn talks hosted by different software companies.
- Participated in creating robots from scratch by having them view and observe human motion.
- Corrected weak code during mock interviews to demonstrate an understanding of algorithmic principles.

New Student Orientation

Orientation Leader

May 2025 – July 2025

- Guided groups of 20+ students and families by being a point of contact.
- Provided information about the campus resources and tools for the students to utilize.
- Collaborated with other orientation leaders to help improve overall experiences for all.

Dowdy Multicultural Center - Peer Mentoring Program

Peer Mentor

August 2024 – May 2026

- Supported students of color to successfully integrate academically and socially into university life.
- Attended other multicultural organization events alongside mentee(s).
- Done weekly meetups for checking on well-being and providing recommended personal resources.

Brothers Of Excellence

Board Member

September 2025 – May 2026

- Connected with industry professionals and alumni to foster networking opportunities.
- Coordinated volunteer efforts for other conferences in partnership with local non-profit organizations.
- Allocate professional development resources and workshops for students to build leadership.

EXPERIENCE

Graduate Assistant

Research Assistant

August 2026 – Present

- Contributed to the Audiovisual Labeled Emotion (ALE) Research Platform, a low-cost system for collecting labeled audiovisual data on human behavior and interaction.
- Collected and organized self-reported affect and activity data in natural educational and human-interaction settings.
- Annotated audiovisual recordings by labeling observed emotions, activities, and interactions to support the development of behavioral research datasets.